



Name: Rahin Ibne Harun

Id: 23-54177-3

Sec: U

Course: Compiler Design Assignment

Final Lab Assignment

Lab Task-4

Task 1:

Write a C++ program that takes the description of a **NFA** and one input

string, then checks whether the string is **Accepted** or **Rejected** by the NFA.

In an NFA, for the same state and symbol, there may be **more than one next state**.

Input format

- First line: two integers n and m
n = number of states, states are numbered from 0 to n-1
m = number of input symbols
- Next m lines: the input symbols
- Next line: integer t, the number of transitions
- Next t lines: current_state input_symbol next_state
- Next line: start state
- Next line: integer f, the number of final states
- Next line: f integers representing final states
- Last line: one input string

Output

Print:

- Accepted if at least one path ends in a final state
- Rejected otherwise

Answer:

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;
```

```
int main() {
```

```
    int n, m;
```

```
    cin >> n >> m;
```

```
    char symbols[100];
```

```
    for (int i = 0; i < m; i++) {
```

```
    cin >> symbols[i];

}

int t;

cin >> t;

int from[200], to[200];

char ch[200];

for (int i = 0; i < t; i++) {

    cin >> from[i] >> ch[i] >> to[i];

}

int start;

cin >> start;

int f;

cin >> f;

int finalStates[100];

for (int i = 0; i < f; i++) {

    cin >> finalStates[i];

}


string input;

cin >> input;


vector<int> currentStates;
```

```
currentStates.push_back(start);
```

```
for (int i = 0; i < input.size(); i++) {
```

```
    vector<int> nextStates;
```

```
    for (int j = 0; j < currentStates.size(); j++) {
```

```
        int curr = currentStates[j];
```

```
        for (int k = 0; k < t; k++) {
```

```
            if (from[k] == curr && ch[k] == input[i]) {
```

```
                nextStates.push_back(to[k]);
```

```
            }
```

```
        }
```

```
    }
```

```
    currentStates = nextStates;
```

```
}
```

```
bool accepted = false;
```

```
for (int i = 0; i < currentStates.size(); i++) {
```

```
    for (int j = 0; j < f; j++) {
```

```
        if (currentStates[i] == finalStates[j]) {
```

```
        accepted = true;
    }
}
}

if (accepted)

    cout << "Accepted";

else

    cout << "Rejected";


return 0;

}
```

Input with output file ss:

```
"E:\Downloads\Compile" × + ▾ − □ ×
3 2
a
b
5
0 a 0
0 a 1
0 b 0
1 b 2
2 a 2
0
2
aab
Accepted
Process returned 0 (0x0)    execution time : 4
9.177 s
Press any key to continue.
|
```

Task 2:

Write a C++ program that takes the description of an **epsilon NFA** and prints:

- the **epsilon closure** of each state
- the **move set** of a given state set for a given input symbol

Use e to represent epsilon transition.

Input format

- First line: two integers n and m
- Next m lines: input symbols
- Next line: integer t, the number of transitions
- Next t lines: current_state input_symbol next_state
- Next line: one integer k, the size of a state set
- Next line: k integers representing the state set
- Next line: one input symbol for move calculation

Output

- Print epsilon closure of every state in set form
- Then print the move result for the given state set and symbol

Answer

```
#include <iostream>
using namespace std;

int n, m, t;
int from[200], to[200];
char ch[200];

bool visited[100];

void eClosure(int state, int closure[], int &size) {
    visited[state] = true;
    closure[size++] = state;

    for (int i = 0; i < t; i++) {
        if (from[i] == state && ch[i] == 'e') {
            if (!visited[to[i]]) {
                eClosure(to[i], closure, size);
            }
        }
    }
}
```

```
}  
}  
}  
}
```

```
int main() {
```

```
    cin >> n >> m;
```

```
    char symbols[100];
```

```
    for (int i = 0; i < m; i++) {
```

```
        cin >> symbols[i];
```

```
    }
```

```
    cin >> t;
```

```
    for (int i = 0; i < t; i++) {
```

```
        cin >> from[i] >> ch[i] >> to[i];
```

```
    }
```

```
    for (int i = 0; i < n; i++) {
```

```
        int closure[100];
```

```
        int size = 0;
```

```
        for (int j = 0; j < n; j++) {
```

```
            visited[j] = false;
```

```
        }
```

```
        eClosure(i, closure, size);
```

```
        cout << "Epsilon closure of " << i << " = {";
```

```
        for (int j = 0; j < size; j++) {
```

```
            cout << closure[j];
```

```
            if (j != size - 1) cout << ",";
```

```
        }
```

```
        cout << "}" << endl;
```

```
    }
```

```

int k;
cin >> k;

int stateSet[100];
for (int i = 0; i < k; i++) {
    cin >> stateSet[i];
}
char inputSymbol;
cin >> inputSymbol;
int moveSet[100];
int moveSize = 0;
for (int i = 0; i < k; i++) {
    int curr = stateSet[i];

    for (int j = 0; j < t; j++) {
        if (from[j] == curr && ch[j] == inputSymbol) {
            bool exists = false;
            for (int x = 0; x < moveSize; x++) {
                if (moveSet[x] == to[j]) {
                    exists = true;
                    break;
                }
            }
            if (!exists) {
                moveSet[moveSize++] = to[j];
            }
        }
    }
}
cout << "Move{";
for (int i = 0; i < k; i++) {
    cout << stateSet[i];
    if (i != k - 1) cout << ",";
}

```

Input with output file ss:

End